



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

BITÁCORA DE LABORATORIO S07

Mexi Banco: directorio, gateway y balanceo

MATERIA

Sistemas Distribuidos

Alumno

Reyes Vázquez Fernando

PROFESOR

Oscar Manuel Ruiz Hurtado

FECHA DE ENTREGA | 10 DE OCTUBRE DE 2026
FECHA DE REALIZACIÓN | 9 DE OCTUBRE DE 2026
GRUPO 02 | SEMESTRE 2027-1

Paso 1 · La historia

Fecha de realización: 9 de octubre de 2026
Hora aproximada de inicio: 17:48

Al inicio, Mexi Banco podía verse como un sistema en el que varias operaciones ocurrían dentro de una misma aplicación. En ese caso, una parte del programa podía llamar directamente a un método de otra parte y recibir la respuesta sin preocuparse por direcciones, puertos o fallas de red.

Al separar el banco en servicios, esa llamada dejó de ser interna y pasó a realizarse por HTTP. Con esto aparecieron problemas que antes no estaban presentes: cada servicio necesita saber dónde está el otro, una petición puede fallar o tardar, los endpoints internos pueden quedar expuestos y, si existen varias copias de un mismo servicio, alguien debe decidir a cuál enviar cada solicitud. La práctica se enfocó en resolver esos puntos mediante un directorio de servicios, un API Gateway y balanceo de carga.

Paso 2 · Directorio

Hora de trabajo: 17:48 - 18:45

Primero se verificó el estado inicial del sistema. La base de datos y los cinco servicios estaban activos y saludables. La consulta directa al servicio CUENTA por el puerto 8081 sí llegaba al servicio, aunque la CLABE 000 no existía y por eso se obtuvo un 404. En cambio, el puerto 8761 todavía rechazaba la conexión porque Eureka aún no estaba habilitado.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
mexi-banco-v06-cuenta-1            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     6 days ago    Up 6 days (healthy)    0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                postgres:17-alpine                 "docker-entrypoint.s..."            db         2 minutes ago Up 2 minutes (healthy) 5432/tcp
mexi-banco-v06-movimiento-1        mexi-banco-v06-movimiento          "java -jar app.jar"                  movimiento 6 days ago    Up 6 days (healthy)    0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1      mexi-banco-v06-notificacion        "java -jar app.jar"                  notificacion 6 days ago    Up 6 days (healthy)    0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1              mexi-banco-v06-spei                "java -jar app.jar"                  spei        6 days ago    Up 6 days (healthy)    0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1     mexi-banco-v06-transferencia       "java -jar app.jar"                  transferencia 6 days ago    Up 6 days (healthy)    0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
```

Figura 1. Estado inicial de Mexi Banco con la base de datos y los cinco servicios activos y en estado saludable.

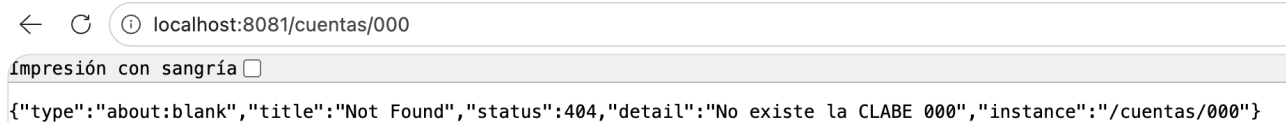


Figura 2. Consulta directa al servicio CUENTA por el puerto 8081; la CLABE solicitada no existe y el servicio responde con código 404.

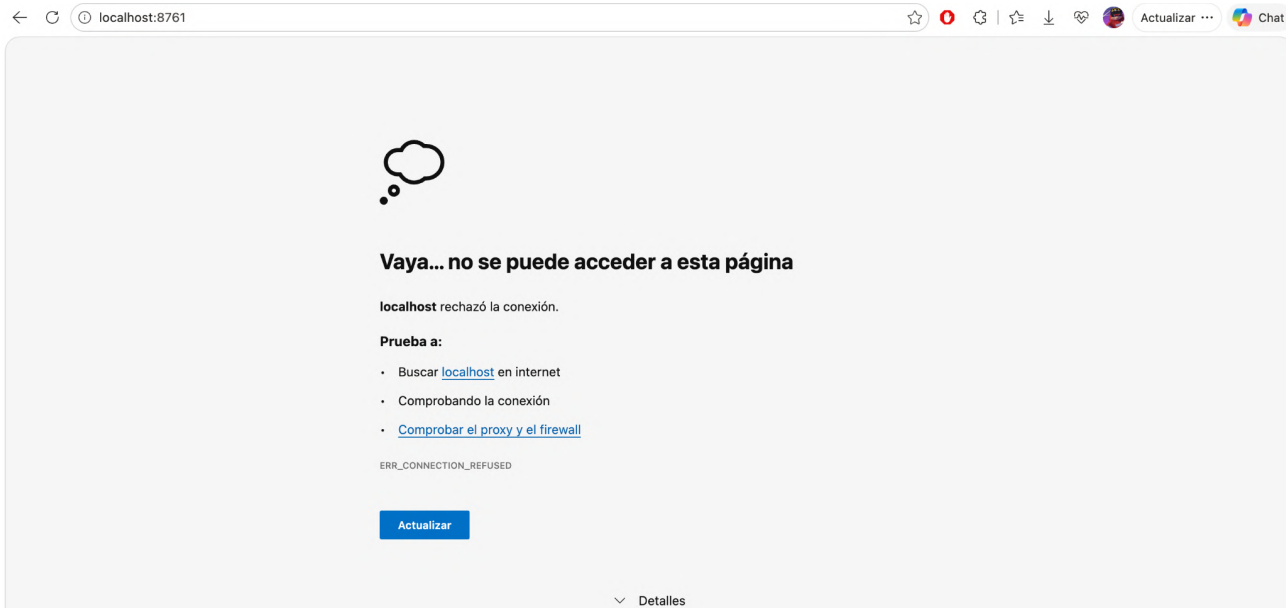


Figura 3. Intento de acceso al puerto 8761 antes de habilitar Eureka; el directorio todavía no se encuentra disponible.

2.1 · Activación del servidor Eureka

En DiscoveryApplication.java se agregó la anotación @EnableEurekaServer junto con su import. Con esto, la aplicación discovery dejó de ser una aplicación Spring Boot normal y comenzó a funcionar como servidor Eureka.

```

discovery > src > main > java > mx > mexibanco > DiscoveryApplication.java
1  package mx.mexibanco;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
6
7  /**
8   * El directorio de Mexi Banco v06. Cada servicio se da de alta aqui con su nombre logico
9   * (cuenta, movimiento, ...) y su IP, y manda un latido cada tanto. Quien quiere hablar con
10  * "cuenta" pregunta aqui que instancias hay, en vez de tener escrita una URL.
11  *
12  * Dashboard: http://localhost:8761
13  */
14  @SpringBootApplication
15  @EnableEurekaServer

```

Figura 4. Modificación de `DiscoveryApplication.java` para habilitar el servidor Eureka.

2.2 · Configuración del directorio

En `application.yml` se desactivó el registro del propio servidor y la descarga del registro mediante `register-with-eureka: false` y `fetch-registry: false`. De esta forma, el directorio funciona como el servidor central de esta práctica y no intenta registrarse consigo mismo.

```

discovery > src > main > resources > application.yml
8  eureka:
9      client:
10         ··· register-with-eureka: false
11         ··· fetch-registry: false

```

Figura 5. Configuración de Eureka para evitar que el directorio se registre y consulte a sí mismo.

2.3 · Registro de los servicios

Después se habilitó la dependencia del cliente de Eureka en los cinco servicios: cuenta, movimiento, transferencia, notificación y SPEI. La misma modificación se realizó en cada `pom.xml` para que todos pudieran registrarse en el directorio al iniciar.

```

cuenta > pom.xml
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
27  <dependencies>
40  <dependency>
43  </dependency>
44  <!-- TODO H1: descomenta el bloque de abajo (borra la linea que abre el comentario y la que lo
45  cierra). Es el cliente de Eureka: al arrancar, el servicio se registra en el directorio
46  con su spring.application.name y manda latidos. Trae tambien el balanceador que usa H3. -->
47
48  <dependency>
49  <groupId>org.springframework.cloud</groupId>
50  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
51  </dependency>
52
53  <dependency>

```

Figura 6. Dependencia del cliente de Eureka habilitada en los servicios; la misma modificación se aplicó en los demás `pom.xml`.

2.4 · Puesta en marcha de discovery

Se habilitó el servicio discovery en `docker-compose.yml` y se reconstruyeron los contenedores. Al terminar, la base de datos, discovery y los cinco servicios aparecieron en estado healthy.

```

=> [speil] Resolving provenance for metadata file
[+] up 13/13
0.0s
✓ Image mexi-banco-v06-spei Built 169.7s
✓ Image mexi-banco-v06-discovery Built 169.7s
✓ Image mexi-banco-v06-cuenta Built 169.7s
✓ Image mexi-banco-v06-movimiento Built 169.7s
✓ Image mexi-banco-v06-transferencia Built 169.7s
✓ Image mexi-banco-v06-notificacion Built 169.7s
✓ Container mexi-banco-v06-db-1 Healthy 3.8s
✓ Container mexi-banco-v06-discovery-1 Healthy 8.8s
✓ Container mexi-banco-v06-movimiento-1 Started 9.0s
✓ Container mexi-banco-v06-spei-1 Started 8.9s
✓ Container mexi-banco-v06-transferencia-1 Started 9.1s
✓ Container mexi-banco-v06-notificacion-1 Started 8.9s
✓ Container mexi-banco-v06-cuenta-1 Started 9.0s
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta      2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                 "docker-entrypoint.s..."            db          58 minutes ago Up 58 minutes (healthy) 5432/tcp
mexi-banco-v06-discovery-1           mexi-banco-v06-discovery            "java -jar app.jar"                  discovery   2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-movimiento-1          mexi-banco-v06-movimiento            "java -jar app.jar"                  movimiento 2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1        mexi-banco-v06-notificacion          "java -jar app.jar"                  notificacion 2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1               mexi-banco-v06-spei                 "java -jar app.jar"                  spei        2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1       mexi-banco-v06-transferencia         "java -jar app.jar"                  transferencia 2 minutes ago Up 2 minutes (healthy) 0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
fernandoreyesvazquez@Air-de-Fernando mexi-banco %

```

Figura 7. Estado de los siete contenedores después de habilitar Eureka, con el directorio y los cinco servicios en estado saludable.

2.5 · Verificación del registro

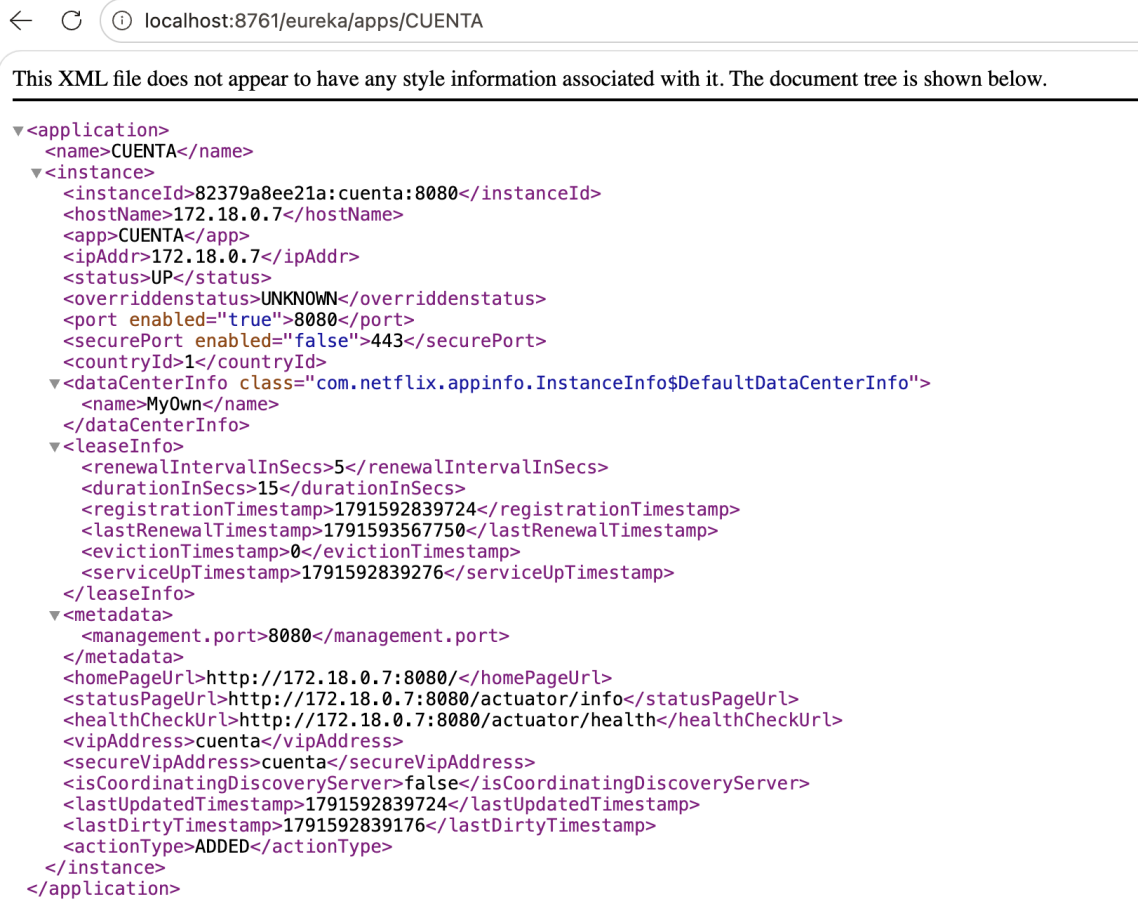
Al abrir localhost:8761, el tablero de Eureka mostró los cinco servicios registrados como UP (1). Esto confirmó que cada servicio estaba enviando sus latidos y que el directorio conocía una instancia de cada uno.

The screenshot shows the Spring Eureka dashboard. At the top, there's a navigation bar with 'HOME' and 'LAST 1000 SINCE STARTUP'. Below this, the 'System Status' section displays environment (test), data center (default), current time (2026-10-10T00:45:33 +0000), uptime (00:05), lease expiration enabled (true), renew threshold (10), and renew interval (120). A red warning message states: 'THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.' Below this, the 'DS Replicas' section shows 'localhost'. The 'Instances currently registered with Eureka' section contains a table with the following data:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 82379a8ee21a:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 94b17aec7085:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 4119835e8e8a:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 08f2a23fca83:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 9074bf32e999:transferencia:8080

Figura 8. Tablero de Eureka con CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA registrados con una instancia activa.

También se consultó el XML de CUENTA. En la información registrada se observó el estado UP, el puerto interno 8080, un `renewalIntervalInSecs` de 5 segundos y un `durationInSecs` de 15 segundos.



```

gateway > src > main > resources > application.yml
4   spring:
7     cloud:
8       gateway:
22      routes:
23        - id: cuenta
24          uri: lb://cuenta
25          predicates:
26            - Path=/api/cuentas,/api/cuentas/{clabe}
27          filters:
28            - StripPrefix=1
29
30        - id: movimiento
31          uri: lb://movimiento
32          predicates:
33            - Path=/api/movimientos
34            - Method=GET
35          filters:
36            - StripPrefix=1
37
38        - id: transferencia
39          uri: lb://transferencia
40          predicates:
41            - Path=/api/transferencias,/api/transferencias/{id}
42          filters:
43            - StripPrefix=1
44
45        - id: notificacion
46          uri: lb://notificacion
47          predicates:
48            - Path=/api/notificaciones
49            - Method=GET
50          filters:
51            - StripPrefix=1
52
53        - id: spei
54          uri: lb://spei
55          predicates:
56            - Path=/api/spei
57            - Method=POST
58          filters:
59            - StripPrefix=1

```

Figura 11. Rutas configuradas en el API Gateway para los servicios de Mexi Banco.

3.2 · Activación del API Gateway

Después se habilitó el servicio gateway en docker-compose.yml, publicado en el puerto 8080 y dependiente de discovery. Al reconstruirlo, el sistema quedó con ocho contenedores saludables y el gateway también apareció registrado en Eureka.

```

docker-compose.yml
49   services:
50
51     # TODO H2: descomenta el servicio gateway (la unica puerta). Antes completa las rutas en
52     # gateway/src/main/resources/application.yml.
53     gateway:
54       build: ../gateway
55       ports:
56         - "8080:8080"
57       environment:
58         <<: *directorio
59       depends_on:
60         discovery:
61           condition: service_healthy
62       healthcheck:
63         test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
64         interval: 5s
65         timeout: 3s
66         retries: 5
67         start_period: 120s

```

Figura 12. Servicio gateway habilitado en docker-compose.yml con publicación en el puerto 8080.

```

=> => naming to docker.io/library/mexi-banco-v06-gateway:latest
=> => unpacking to docker.io/library/mexi-banco-v06-gateway:latest
=> [gateway] resolving provenance for metadata file
[+] up 4/4
✓ Image mexi-banco-v06-gateway Built 19.8s
✓ Image mexi-banco-v06-discovery Built 19.8s
✓ Container mexi-banco-v06-discovery-1 Healthy 0.6s
✓ Container mexi-banco-v06-gateway-1 Started 0.9s
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED      STATUS      PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta             "java -jar app.jar"                  cuenta     42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8080
1->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                "docker-entrypoint.s..."            db         2 hours ago Up 2 hours (healthy) 5432/tcp
mexi-banco-v06-discovery-1          sha256:771f06403ced81c1b53d98ad1da9dfc2207ca8c69a700608d9b01db708f6d441 discovery 42 minutes ago Up 42 minutes (healthy) 0.0.0.0:876
1->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1            mexi-banco-v06-gateway            "java -jar app.jar"                  gateway    22 seconds ago Up 21 seconds (healthy) 0.0.0.0:8080
0->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento         "java -jar app.jar"                  movimiento 42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8080
2->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion       "java -jar app.jar"                  notificacion 42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8080
4->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1              mexi-banco-v06-spei              "java -jar app.jar"                  spei       42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8080
5->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia      "java -jar app.jar"                  transferencia 42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8080
3->8080/tcp, [::]:8083->8080/tcp

```

Figura 13. Estado de los servicios después de habilitar el API Gateway, mostrando los ocho contenedores activos y saludables.

The screenshot shows the Spring Eureka dashboard in a web browser. The top navigation bar includes the Spring Eureka logo and a 'HOME' link. The main content area is divided into several sections:

- System Status:** A table showing environment details (test, default) and current time (2026-10-10T01:24:11 +0000). It also displays uptime (00:44), lease expiration enabled (true), renew threshold (0), and renew interval (130).
- THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.**
- DS Replicas:** A section showing the status of the data source replicas.
- Instances currently registered with Eureka:** A table listing the following applications and their status:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 82379a8ee21a:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 780edd921a9f:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 94b17aec7085:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 4119835a8e8a:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 08f2a23fca83:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 9074bf32e999:transferencia:8080
- General Info:** A section showing the total available memory (82mb) and the number of instances (0).

Figura 14. Tablero de Eureka después de habilitar el API Gateway, mostrando GATEWAY junto con los cinco servicios.

3.3 · Pruebas por la puerta única

Se realizaron dos pruebas que devolvieron 404, pero por causas diferentes. Al consultar `/api/cuentas/000` apareció la cabecera `X-Instancia`, lo que demuestra que el gateway sí encontró la ruta, consultó el directorio y la petición llegó al servicio CUENTA; el 404 fue generado por CUENTA porque la CLABE no existía. En cambio, el POST a `/api/movimientos` también devolvió 404, pero sin `X-Instancia`, porque el gateway lo detuvo al no existir una ruta que permita esa operación desde afuera.

```

fernandoreyesvazquez@Air-de-Fernando mexi-banco % curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 82379a8ee21a
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 01:30:15 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
fernandoreyesvazquez@Air-de-Fernando mexi-banco % curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T01:30:28.068+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"db79e31d-93"}

```

Figura 15. Dos respuestas 404 por causas distintas: la consulta de CUENTA llega al servicio y contiene X-Instancia, mientras que el POST a MOVIMIENTO es rechazado por el gateway.

Finalmente se creó una cuenta a través del gateway. El alta respondió 201 Created y la consulta posterior respondió 200 OK con saldo inicial de cero. La URL quedó guardada para utilizarla en los ejercicios de balanceo y caída.

```

fernandoreyesvazquez@Air-de-Fernando mexi-banco % LAB_CLABE="S07$(date +%s)"
fernandoreyesvazquez@Air-de-Fernando mexi-banco % curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 82379a8ee21a
Content-Type: application/json
Date: Sat, 10 Oct 2026 01:35:18 GMT

[{"id":1,"clabe":"S071791596053","titular":"S07","saldo":0}]
fernandoreyesvazquez@Air-de-Fernando mexi-banco % URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
fernandoreyesvazquez@Air-de-Fernando mexi-banco % curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 82379a8ee21a
Content-Type: application/json
Date: Sat, 10 Oct 2026 01:35:47 GMT

[{"id":1,"clabe":"S071791596053","titular":"S07","saldo":0.00}]
fernandoreyesvazquez@Air-de-Fernando mexi-banco % echo "$URL"
http://localhost:8080/api/cuentas/S071791596053

```

Figura 16. Alta y consulta de una cuenta mediante el API Gateway: creación con código 201 y consulta posterior con código 200.

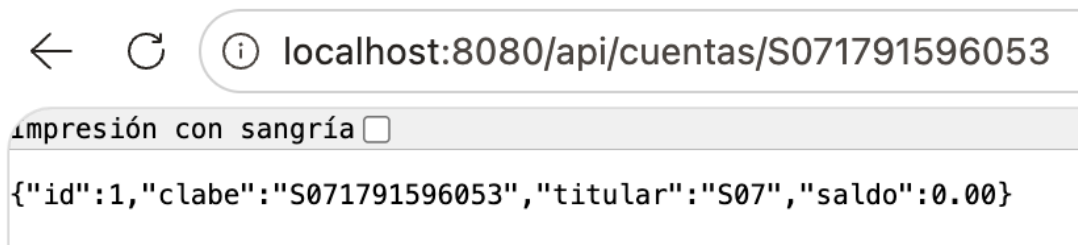


Figura 17. Consulta de la cuenta creada desde el navegador utilizando únicamente el puerto 8080 del gateway.

Pasos 4 y 5 · Copias y caída

Hora de trabajo: 19:39 - 20:36

4.1 · Tres copias de CUENTA

El servicio CUENTA se escaló a tres réplicas mediante deploy: replicas: 3. Para evitar conflictos de puertos, se retiró la publicación directa del puerto 8081. También se eliminaron los puertos publicados de movimiento, transferencia, notificación y SPEI, de modo que hacia la máquina anfitriona solo quedaron expuestos el gateway en 8080 y Eureka en 8761.

```

105 # Cuentas y saldos. Llama a movimiento para asentar el saldo inicial.
106 cuenta:
107   <<: *servicio
108   build: ./cuenta
109   # TODO H3: cambia estas dos lineas de `ports` por tres replicas:
110   #   deploy:
111   #     replicas: 3
112   # Tres contenedores no pueden publicar el mismo puerto 8081 de tu maquina, y ya no hace falta:
113   # desde afuera se llega por el gateway (H2).
114
115   # antes: una copia, publicada en el 8081
116   #ports:
117   # - "8081:8080"
118
119   # después: tres copias, sin puerto publicado
120   deploy:
121     replicas: 3

```

Figura 18. Configuración de CUENTA con tres réplicas y sin puerto 8081 publicado.

```

49 services:
127 # El ledger. No llama a nadie.
128 movimiento:
129   <<: *servicio
130   build: ./movimiento
131   #ports:
132   # - "8082:8080"
133   environment:
134     <<: *entorno-base
135     DB_NAME: movimiento
136
137 # Transferencias internas. Orquesta a cuenta, movimiento y notificacion por HTTP.
138 transferencia:
139   <<: *servicio
140   build: ./transferencia
141   #ports:
142   # - "8083:8080"
143   environment:
144     <<: *entorno-base
145     DB_NAME: transferencia
146     # TODO H3: llamar por NOMBRE, sin puerto: http://cuenta, http://movimiento y
147     # http://notificacion. Solo funciona con el RestClient.Builder @LoadBalanced (TODO H3 en
148     # transferencia/.../clientes/ClientesHttp.java); sin el, "cuenta" se resuelve por DNS al
149     # puerto 80 y la transferencia responde 503.
150     CUENTA_URL: http://cuenta:8080
151     MOVIMIENTO_URL: http://movimiento:8080
152     NOTIFICACION_URL: http://notificacion:8080
153     # Solo para la demo de "lo que se rompio al partir" (ver README). 0 = sin pausa.
154     PAUSA_ENTRE_CARGO_Y_ABONO_MS: ${PAUSA_ENTRE_CARGO_Y_ABONO_MS:-0}
155
156 # Avisos al cliente. No llama a nadie.
157 notificacion:
158   <<: *servicio
159   build: ./notificacion
160   #ports:
161   # - "8084:8080"
162   environment:
163     <<: *entorno-base
164     DB_NAME: notificacion
165
166 # Transferencia a otro banco con Idempotency-Key. Llama a cuenta, movimiento y notificacion.
167 spei:
168   <<: *servicio
169   build: ./spei
170   #ports:
171   # - "8085:8080"

```

Figura 19. Puertos directos de los servicios internos deshabilitados en docker-compose.yml.

```

fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose up -d
[+] up 10/10
✓ Container mexi-banco-v06-gateway-1      Running
✓ Container mexi-banco-v06-discovery-1    Healthy
✓ Container mexi-banco-v06-db-1           Healthy
✓ Container mexi-banco-v06-cuenta-1       Started
✓ Container mexi-banco-v06-notificacion-1 Started
✓ Container mexi-banco-v06-cuenta-3       Started
✓ Container mexi-banco-v06-spei-1         Started
✓ Container mexi-banco-v06-cuenta-2       Started
✓ Container mexi-banco-v06-transferencia-1 Started
✓ Container mexi-banco-v06-movimiento-1   Started
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
7d1f10f1aa10 mexi-banco-v06-cuenta-1 8080/tcp
aba2051306bf mexi-banco-v06-cuenta-2 8080/tcp
1a2eeaf08ac0 mexi-banco-v06-cuenta-3 8080/tcp

```

Figura 20. Tres contenedores de CUENTA ejecutándose de forma simultánea, cada uno escuchando internamente en 8080/tcp.

Eureka registró las tres copias de CUENTA como UP (3). Los identificadores mostrados en el tablero coincidieron con los IDs observados en Docker.

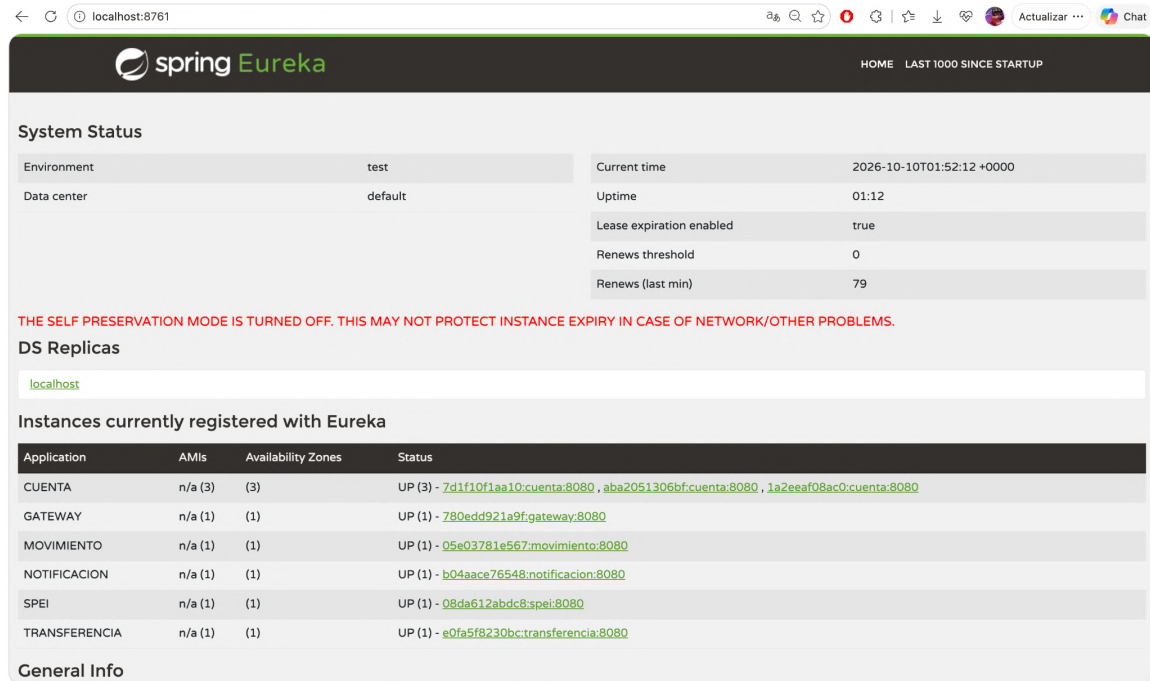


Figura 21. Tablero de Eureka después de escalar CUENTA, mostrando tres instancias activas registradas como UP (3).

4.2 · Balanceo round robin

Se enviaron nueve consultas consecutivas a la misma cuenta a través del gateway. La cabecera X-Instancia mostró tres identificadores que se repitieron por turnos, lo que permitió comprobar el balanceo round robin entre cuenta-1, cuenta-2 y cuenta-3.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco % for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: aba2051306bf
X-Instancia: 7d1f10f1aa10
X-Instancia: 1a2eeaf08ac0
X-Instancia: aba2051306bf
X-Instancia: 7d1f10f1aa10
X-Instancia: 1a2eeaf08ac0
X-Instancia: aba2051306bf
X-Instancia: 7d1f10f1aa10
X-Instancia: 1a2eeaf08ac0
```

Figura 22. Nueve solicitudes a CUENTA distribuidas entre las tres réplicas mediante round robin.

```
/Users/fernandoreyesvazquez/Documents/SD_Clase/mexi-banco
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose logs -f cuenta | grep atendida
cuenta-1 | 2026-10-10T01:56:01.826Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-1 | 2026-10-10T01:56:03.302Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-1 | 2026-10-10T01:56:03.508Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-3 | 2026-10-10T01:56:02.474Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-3 | 2026-10-10T01:56:03.420Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-3 | 2026-10-10T01:56:03.523Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-2 | 2026-10-10T01:56:00.560Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por aba2051306bf
cuenta-2 | 2026-10-10T01:56:03.201Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por aba2051306bf
cuenta-2 | 2026-10-10T01:56:03.474Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por aba2051306bf
cuenta-2 | 2026-10-10T02:06:50.173Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-3 | 2026-10-10T02:06:50.378Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-1 | 2026-10-10T02:06:50.540Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-2 | 2026-10-10T02:06:50.688Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por aba2051306bf
cuenta-3 | 2026-10-10T02:06:50.729Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-1 | 2026-10-10T02:06:50.770Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
cuenta-2 | 2026-10-10T02:06:50.843Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por aba2051306bf
cuenta-3 | 2026-10-10T02:06:50.891Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 1a2eeaf08ac0
cuenta-1 | 2026-10-10T02:06:50.917Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791596053 atendida por 7d1f10f1aa10
```

Figura 23. Logs de las tres réplicas de CUENTA mostrando solicitudes atendidas por cuenta-1, cuenta-2 y cuenta-3.

5 · Caída de una réplica

Predicción: Pienso que no fallarán todas las solicitudes, sino solo algunas, porque todavía quedarán dos copias de CUENTA funcionando. Durante unos segundos el gateway podría seguir enviando peticiones a la instancia que se apagó, por lo que espero ver algunos errores hasta que Eureka detecte la caída y actualice las instancias disponibles.

Mientras se ejecutaba una consulta por segundo, se provocó la caída abrupta de cuenta-1 mediante docker kill. Antes de ejecutar el comando se registró la hora del apagado y, a partir de ese momento, se continuó observando tanto la respuesta de las peticiones como el tablero de Eureka para identificar cuándo el sistema detectaba la caída..

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco % date +%T
[20:25:16
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
fernandoreyesvazquez@Air-de-Fernando mexi-banco % date +%T
[20:25:50
```

Figura 24. Hora y comando utilizados para provocar la caída abrupta de cuenta-1.

```
echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
sleep 1
[done
20:25:14 200
20:25:15 200
20:25:17 200
20:25:18 200
20:25:19 200
20:25:20 500
20:25:23 200
20:25:25 200
20:25:26 500
20:25:29 200
20:25:30 200
20:25:31 200
20:25:32 200
20:25:33 500
20:25:37 500
20:25:40 200
20:25:41 200
20:25:42 200
20:25:43 200
20:25:44 500
20:25:47 200
20:25:49 200
20:25:50 500
20:25:53 200
20:25:54 200
20:25:55 200
20:25:56 200
20:25:57 200
20:25:59 200
20:26:00 200
20:26:01 200
20:26:02 200
20:26:03 200
20:26:04 200
20:26:05 200
20:26:06 200
20:26:08 200
20:26:09 200
20:26:10 200
20:26:11 200
20:26:12 200
20:26:13 200
20:26:14 200
20:26:15 200
20:26:17 200
20:26:18 200
20:26:19 200
20:26:20 200
20:26:21 200
20:26:22 200
20:26:23 200
20:26:24 200
20:26:25 200
20:26:27 200
20:26:28 200
20:26:29 200
20:26:30 200
20:26:31 200
20:26:33 200
20:26:34 200
```

Figura 25. Respuestas obtenidas después de la caída; se observan cinco errores HTTP 500 intermitentes entre respuestas 200.

The screenshot shows the Spring Eureka web interface at localhost:8761. The header includes the Spring Eureka logo and navigation links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

- System Status:** A table showing environment (test), data center (default), current time (2026-10-10T02:26:34 +0000), uptime (01:46), lease expiration enabled (true), renew threshold (0), and renews (last min) (168).
- DS Replicas:** A section showing the local host (localhost).
- Instances currently registered with Eureka:** A table listing applications and their status.
- General Info:** A section for additional information.

Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - aba2051306bfr:cuenta:8080 , 1a2eeaf08ac0:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 780edd921a9f:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 05e03781e567:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - b04aace76548:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 08da612abdc8:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - e0fa5f8230bc:transferencia:8080

Figura 26. Eureka después de retirar la instancia caída del directorio, quedando CUENTA en estado UP (2).

El apagado se realizó a las 20:25:16 y Eureka cambió de UP (3) a UP (2) a las 20:25:50. El tiempo transcurrido fue de 34 segundos. Durante ese intervalo se observaron cinco errores HTTP 500; el último apareció a las 20:25:50. Esto ocurrió porque el gateway todavía tenía temporalmente información de la instancia caída mientras Eureka y las cachés de descubrimiento actualizaban la lista de instancias disponibles.

Hora del apagado: 20:25:16

Cambio de UP (3) a UP (2): 20:25:50

Tiempo transcurrido: 34 segundos

Errores observados: 5

Tipo de error: HTTP 500

Último error: 20:25:50

Después se volvió a iniciar cuenta-1. La instancia se registró nuevamente en Eureka y CUENTA regresó a UP (3), recuperando las tres copias activas.

```
[fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

Figura 27. Comando utilizado para iniciar nuevamente cuenta-1.

The screenshot shows the Spring Eureka web interface at localhost:8761. The top navigation bar includes 'HOME' and 'LAST 1000 SINCE STARTUP'. The 'System Status' section displays the following information:

Environment	test	Current time	2026-10-10T02:36:42 +0000
Data center	default	Uptime	01:56
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	168

A red warning message states: 'THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.'

The 'DS Replicas' section shows 'localhost'.

The 'Instances currently registered with Eureka' section contains a table with the following data:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 7d1f10f1aa10:cuenta:8080 , aba2051306bf:cuenta:8080 , 1a2eeaf08ac0:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 780edd921a9f:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 05e03781e567:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - b04aace76548:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 08da612abdc8:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - e0fa5f8230bc:transferencia:8080

The 'General Info' section is also visible at the bottom.

Figura 28. Recuperación de cuenta-1 en Eureka y restablecimiento de CUENTA en UP (3).

Cierre

Hora aproximada: 20:39

El directorio resuelve cómo localizar los servicios y saber qué instancias están disponibles sin depender de direcciones escritas manualmente.

El gateway resuelve el acceso al sistema mediante una sola entrada y permite controlar qué operaciones pueden realizarse desde el exterior.

El balanceador resuelve el reparto de las solicitudes entre varias copias de un mismo servicio.

Cuando murió una copia vi cinco errores HTTP 500 intermitentes, porque el gateway todavía intentó enviar algunas solicitudes a la instancia caída mientras Eureka y sus cachés actualizaban la lista de instancias disponibles.

Tarea · Balanceo interno en transferencia

Hora de trabajo: 20:43 - 21:05

La tarea consistió en hacer que el servicio transferencia también utilizara el directorio y el balanceador al llamar a CUENTA. Hasta ese momento, el gateway ya repartía las consultas externas, pero transferencia seguía utilizando direcciones con puerto fijo.

T1 · Cliente HTTP balanceado

En ClientesHttp.java se agregaron RestClientBuilderConfigurer y @LoadBalanced. Después se declaró un RestClient.Builder balanceado para que nombres como http://cuenta fueran interpretados como nombres de servicio registrados en Eureka.

```
transferencia > src > main > java > mx > mexibanco > clientes > ClientesHttp.java
11 import org.springframework.http.client.SimpleClientHttpRequestFactory;
12 import org.springframework.web.client.RestClient;
13 import org.springframework.boot.autoconfigure.web.client.RestClientBuilderConfigurer;
14 import org.springframework.cloud.client.loadbalancer.LoadBalanced;
15
```

Figura 29. Imports agregados en ClientesHttp.java para configurar el cliente balanceado.

```

transferencia > src > main > java > mx > mexibanco > clientes > ClientesHttp.java
30  public class ClientesHttp {
45
46      // TODO H3: declara aqui el RestClient.Builder balanceado que usaran los tres clientes. Es un
47      // metodo @Bean con la anotacion @LoadBalanced (org.springframework.cloud.client.loadbalancer),
48      // que recibe un RestClientBuilderConfigurer (org.springframework.boot.autoconfigure.web.client)
49      // y regresa configurador.configure(RestClient.builder()). El configurador le aplica lo mismo que
50      // Spring Boot le pone a su builder, incluidos los tiemposMaximos de arriba.
51      // Con @LoadBalanced, el host de la URL (http://cuenta) se toma como NOMBRE de servicio y se
52      // cambia, en cada peticion, por la IP y el puerto de una instancia registrada en Eureka.
53
54      @Bean
55      @LoadBalanced
56      RestClient.Builder restClientBalanceado(RestClientBuilderConfigurer configurador) {
57          return configurador.configure(RestClient.builder());
58      }
59

```

Figura 30. *RestClient.Builder declarado como Bean con la anotación @LoadBalanced.*

T2 ·URLs por nombre de servicio

En docker-compose.yml se retiró :8080 de CUENTA_URL, MOVIMIENTO_URL y NOTIFICACION_URL. Las URLs quedaron únicamente con el nombre lógico del servicio para que el cliente balanceado resolviera la instancia mediante Eureka.

```

docker-compose.yml
49  services:
137      # Transferencias internas. Orquesta a cuenta, movimiento y notificacion por HTTP.
138      transferencia:
139          <<: *servicio
140          build: ../transferencia
141          #ports:
142          # - "8083:8080"
143          environment:
144              <<: *entorno-base
145              DB_NAME: transferencia
146              # TODO H3: llamar por NOMBRE, sin puerto: http://cuenta, http://movimiento y
147              # http://notificacion. Solo funciona con el RestClient.Builder @LoadBalanced (TODO H3 en
148              # transferencia/.../clientes/ClientesHttp.java); sin el, "cuenta" se resuelve por DNS al
149              # puerto 80 y la transferencia responde 503.
150              CUENTA_URL: http://cuenta
151              MOVIMIENTO_URL: http://movimiento
152              NOTIFICACION_URL: http://notificacion
153              # Solo para la demo de "lo que se rompio al partir" (ver README). 0 = sin pausa.
154              PAUSA_ENTRE_CARGO_Y_ABONO_MS: ${PAUSA_ENTRE_CARGO_Y_ABONO_MS:-0}

```

Figura 31. *Configuración anterior de transferencia con direcciones que incluían el puerto 8080.*

```

docker-compose.yml
49  services:

138  transferencia:
139    <<: *servicio
140    build: ./transferencia
141    #ports:
142    # - "8083:8080"
143    environment:
144      <<: *entorno-base
145      DB_NAME: transferencia
146      # TODO H3: llamar por NOMBRE, sin puerto: http://cuenta, http://movimiento y
147      # http://notificacion. Solo funciona con el RestClient.Builder @LoadBalanced (TODO H3 en
148      # transferencia/.../clientes/ClientesHttp.java); sin el, "cuenta" se resuelve por DNS al
149      # puerto 80 y la transferencia responde 503.
150      CUENTA_URL: http://cuenta
151      MOVIMIENTO_URL: http://movimiento
152      NOTIFICACION_URL: http://notificacion
153      # Solo para la demo de "lo que se rompio al partir" (ver README). 0 = sin pausa.
154      PAUSA_ENTRE_CARGO_Y_ABONO_MS: ${PAUSA_ENTRE_CARGO_Y_ABONO_MS:-0}

```

Figura 32. Configuración final usando solamente los nombres de servicio: cuenta, movimiento y notificación.

T3 · Reconstrucción y prueba final

Se reconstruyó el servicio transferencia y se verificó que regresara a estado healthy. Después se ejecutó ./demo-v06.sh para comprobar la transferencia, el SPEI, la idempotencia, el balanceo y los casos de error.

```

fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose up -d --build transferencia
[+] Building 24.0s (31/31) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 1.16kB                                         0.0s
=> [transferencia internal] load build definition from Dockerfile        0.1s
=> => transferring dockerfile: 440B                                       0.0s
=> [discovery internal] load build definition from Dockerfile           0.1s
=> => transferring dockerfile: 490B                                       0.0s
=> [transferencia internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine 1.5s

```

Figura 33. Reconstrucción del servicio transferencia después de habilitar el cliente balanceado.

La demostración creó las cuentas de Ana y Beto, transfirió 200 y dejó los saldos en 800 y 700, respectivamente. Después se ejecutó un SPEI de 50 con una Idempotency-Key repetida; el saldo de Ana quedó en 750, lo que confirmó que no se cobró dos veces. También se observaron nuevamente las tres réplicas en round robin y se verificó que los errores 404 y 422 no modificaran los saldos.

```

fernandoreyesvazquez@Air-de-Fernando mexi-banco % ./demo-v06.sh

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/sureka/apps):
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009160091801","titular":"Ana","saldoInicial":1000}'
{"id": 2,
 "clave": "002180009160091801",
 "titular": "Ana",
 "saldo": 1000
}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009160091802","titular":"Beto","saldoInicial":500}'
{"id": 3,
 "clave": "002180009160091802",
 "titular": "Beto",
 "saldo": 500
}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160091801","claveDestino":"002180009160091802","monto":200,"fecha":"2026-10-10T02:56:28.889314845Z"}'
{"id": 1,
 "claveOrigen": "002180009160091801",
 "claveDestino": "002180009160091802",
 "monto": 200,
 "fecha": "2026-10-10T02:56:28.889315Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-1 GET /cuentas/002180009160091801 atendida por 7d1f10f1aa10
cuenta-2 GET /cuentas/002180009160091802 atendida por ab20513060f
cuenta-3 POST /cuentas/002180009160091801/cargos atendida por 1a2eeaf08ac8
cuenta-1 POST /cuentas/002180009160091802/abonos atendida por 7d1f10f1aa10
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/1
{"id": 1,
 "claveOrigen": "002180009160091801",
 "claveDestino": "002180009160091802",
 "monto": 200,
 "fecha": "2026-10-10T02:56:28.889315Z"}
[HTTP 200]

```

Figura 34. Inicio de demo-v06.sh: registro de servicios, creación de las cuentas y transferencia inicial mediante el gateway.

```

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009160091801
{
  "id": 2,
  "clabe": "002180009160091801",
  "titular": "Ana",
  "saldo": 800.00
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009160091802
{
  "id": 3,
  "clabe": "002180009160091802",
  "titular": "Beto",
  "saldo": 700.00
}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clabe=002180009160091801
[
  {
    "id": 3,
    "clabeCuenta": "002180009160091801",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200.00,
    "saldoResultante": 800.00,
    "fecha": "2026-10-10T02:55:28.819234Z",
    "referencia": "a 002180009160091802"
  },
  {
    "id": 1,
    "clabeCuenta": "002180009160091801",
    "tipo": "DEPOSITO",
    "monto": 1000.00,
    "saldoResultante": 1000.00,
    "fecha": "2026-10-10T02:55:23.072493Z",
    "referencia": "saldo inicial"
  }
]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160091802
[
  {
    "id": 2,
    "clabeCuenta": "002180009160091802",
    "mensaje": "TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00",
    "fecha": "2026-10-10T02:55:31.609679Z"
  }
]
[HTTP 200]

```

Figura 35. Comprobación de saldos después de la transferencia y continuación de las pruebas de la demo.

```

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91600918' -d '{"clabeOrigen":"002180009160091801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91600918",
  "clabeOrigen": "002180009160091801",
  "bancoDestino": "BANCO-X",
  "claveDestino": "0121800000000000009",
  "monto": 50,
  "estado": "ENVIADO"
}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91600918' -d '{"clabeOrigen":"002180009160091801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91600918",
  "clabeOrigen": "002180009160091801",
  "bancoDestino": "BANCO-X",
  "claveDestino": "0121800000000000009",
  "monto": 50.00,
  "estado": "ENVIADO"
}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: 1a2eeaf08ac8 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: 7d1f10f1aa10 (mexi-banco-v06-cuenta-1)
consulta 3 X-Instancia: aba2051306bf (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: 1a2eeaf08ac8 (mexi-banco-v06-cuenta-3)
consulta 5 X-Instancia: 7d1f10f1aa10 (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: aba2051306bf (mexi-banco-v06-cuenta-2)
consulta 7 X-Instancia: 1a2eeaf08ac8 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 7d1f10f1aa10 (mexi-banco-v06-cuenta-1)
consulta 9 X-Instancia: 1a2eeaf08ac8 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 0 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009160091801/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{
  "timestamp": "2026-10-10T02:55:38.445+00:00",
  "path": "/api/cuentas/002180009160091801/abonos",
  "status": 404,
  "error": "Not Found",
  "requestId": "ee031e2f-1190"
}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"clabeOrigen":"002180009160091801","claveDestino":"002180009160091899","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009160091899",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"clabeOrigen":"002180009160091801","claveDestino":"002180009160091802","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009160091801",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco
Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

```

Figura 36. Prueba de SPEI idempotente y nueve consultas round robin durante demo-v06.sh.

```

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"clabeOrigen":"002180009160091801","claveDestino":"002180009160091899","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009160091899",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"clabeOrigen":"002180009160091801","claveDestino":"002180009160091802","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009160091801",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco
Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

```

Figura 37. Casos de error 404 y 422; los saldos se mantienen en Ana 750 y Beto 700.

Finalmente, los logs de CUENTA mostraron que los cargos y abonos internos fueron atendidos por distintas réplicas. Esto confirmó que el balanceo ya no solo ocurría en las consultas que entraban por el gateway, sino

también en las llamadas realizadas desde transferencia.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-2 | 2026-10-10T02:55:38.685Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160091801/cargos atendida por aba2051306bf
cuenta-3 | 2026-10-10T02:55:26.883Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160091801/cargos atendida por 1a2eeaf08ac0
cuenta-3 | 2026-10-10T02:55:36.367Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160091801/cargos atendida por 1a2eeaf08ac0
cuenta-1 | 2026-10-10T02:55:28.107Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009160091802/abonos atendida por 7d1f10f1aa10
```

Figura 38. Registros de CUENTA después de habilitar el balanceo interno; cargos y abonos fueron atendidos por distintas réplicas.

Finalización del entorno

Al terminar se ejecutó `docker compose down` sin la opción `-v`. Los contenedores y la red del proyecto fueron retirados, pero los datos se conservaron en los volúmenes para no perder el estado de la práctica.

```
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose down
[+] down 11/11
✓ Container mexi-banco-v06-movimiento-1 Removed 4.2s
✓ Container mexi-banco-v06-notificacion-1 Removed 4.1s
✓ Container mexi-banco-v06-cuenta-1 Removed 4.2s
✓ Container mexi-banco-v06-cuenta-2 Removed 4.2s
✓ Container mexi-banco-v06-transferencia-1 Removed 4.2s
✓ Container mexi-banco-v06-gateway-1 Removed 4.2s
✓ Container mexi-banco-v06-cuenta-3 Removed 4.2s
✓ Container mexi-banco-v06-spei-1 Removed 4.2s
✓ Container mexi-banco-v06-discovery-1 Removed 1.2s
✓ Container mexi-banco-v06-db-1 Removed 0.8s
✓ Network mexibanco-v06 Removed 0.1s
fernandoreyesvazquez@Air-de-Fernando mexi-banco % docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
fernandoreyesvazquez@Air-de-Fernando mexi-banco %
```

Figura 39. Cierre del entorno con `docker compose down` y verificación de que no quedaron contenedores activos del proyecto.